Category C · Coding Assignment

Problem 8: Duty Roster Rotation — Solution

TASK

A hostel rotates its cleaning duty roster every week by moving each name *k* places to the right; names that go past the end wrap around to the front. Implement a function rotateRoster(names, k) that returns the roster after rotating it to the right by *k* places. *k* may be larger than the number of names.

CONSTRAINTS

• 1 ≤ length of names ≤ 100
• $0 \le k \le 10^9$

EXAMPLES

Sample 1: names = [A, B, C, D, E], k = 2
Expected Output: [D, E, A, B, C]

Sample 2: names = [A, B, C, D, E], k = 7
Expected Output: [D, E, A, B, C] *(7 places is the same as 2)*

IMPLEMENTATION (Python)

```
def rotateRoster(names, k):
    n = len(names)
    if n == 0 or k == 0:
        return list(names)

    # Remove full redundant turns
    k = k % n
    if k == 0:
        return list(names)

    # Rotate using list slicing
    return names[-k:] + names[:-k]
```

COMPLEXITY REQUIREMENTS & ANALYSIS

• **Time Complexity:** *O(n)*, where *n* is the number of names in the roster. Computing *k % n* takes *O(1)* time, and list slicing and concatenation take *O(n)* time.

• **Space Complexity:** *O(n)* to store and return the newly rotated roster list.

WHY ROTATING ONE PLACE AT A TIME, k TIMES, IS SLOW WHEN k IS LARGE

If we rotate the roster one position to the right iteratively *k* times:

1. Each single-position shift operation takes $O(n)$ time (shifting elements or moving the last element to the front).

2. Repeating this step k times results in a total time complexity of $O(n \times k)$.

3. Given that k can be as large as 10^9, doing $n \times k$ operations would lead to up to 10^{11} operations, causing a severe Time Limit Exceeded (TLE) error.

4. By applying modulo arithmetic ($k = k \% n$), we limit the effective rotations to at most $n - 1$, ensuring optimal performance well within time limits.